

HOMework-6

REPORT

CSE 421

Efe Kayadelen

Student ID : 20220701088

Samet Alper Özdemir

Student ID : 20210702021

GITHUB REPO: <https://github.com/blackwh1p/CSE421-Homeworks>

ResNet:

resnet.py Code:

This code implements a ResNet v1 architecture using TensorFlow/Keras, specifically designed for efficient image classification. It features a modular `resnet_layer` function that handles the standard Convolution-BatchNormalization-Activation sequence with L_2 regularization. The main ResNet builder uses residual "skip connections" to prevent vanishing gradients, allowing for deeper networks. It automatically organizes layers into three stages of increasing filter depth, includes an optional dropout layer for regularization, and dynamically adjusts the final output layer for either binary or multi-class classification tasks.

Code:

```
# /*-----
# * Copyright (c) 2022 STMicroelectronics.
# * All rights reserved.
# *
# * This software is licensed under terms that can be found in the LICENSE file in
# * the root directory of this software component.
# * If no LICENSE file comes with this software, it is provided AS-IS.
# *-----*/

import tensorflow as tf
from tensorflow.keras.models import Model
from tensorflow.keras import layers
from tensorflow.keras.regularizers import l2
from typing import Tuple

def resnet_layer(inputs: layers.Input,
                 num_filters: int = 16,
                 kernel_size: int = 3,
                 strides: int = 1,
                 activation: str = 'relu',
                 batch_normalization: bool = True,
                 conv_first: bool = True) -> layers.Activation:
    """
    2D Convolution-Batch Normalization-Activation stack builder for ResNet models.
    """
    conv = layers.Conv2D(num_filters,
                        kernel_size=kernel_size,
                        strides=strides,
                        padding='same',
                        kernel_initializer='he_normal',
                        kernel_regularizer=l2(1e-4))

    x = inputs
    if conv_first:
        x = conv(x)
        if batch_normalization:
            x = layers.BatchNormalization()(x)
        if activation is not None:
            x = layers.Activation(activation)(x)
    else:
        if batch_normalization:
            x = layers.BatchNormalization()(x)
```

```

        if activation is not None:
            x = layers.Activation(activation)(x)
        x = conv(x)
    return x

def ResNet(input_shape: Tuple[int, int, int], classes: int = 10, depth: int = 8, dropout: float = None) -> Model:
    """
    ResNet v1 builder.
    """
    if (depth - 2) % 6 != 0:
        raise ValueError('depth should be 6n+2 (eg 20, 32, 44)')

    num_filters = 16
    num_res_blocks = int((depth - 2) / 6)

    inputs = layers.Input(shape=input_shape)
    x = resnet_layer(inputs=inputs)

    # Instantiate the stack of residual units
    for stack in range(3):
        for res_block in range(num_res_blocks):
            strides = 1
            if stack > 0 and res_block == 0: # first layer but not first stack
                strides = 2 # downsample
            y = resnet_layer(inputs=x,
                             num_filters=num_filters,
                             strides=strides)
            y = resnet_layer(inputs=y,
                             num_filters=num_filters,
                             activation=None)
            if stack > 0 and res_block == 0: # first layer but not first stack
                # linear projection residual shortcut connection to match changed dims
                x = resnet_layer(inputs=x,
                                 num_filters=num_filters,
                                 kernel_size=1,
                                 strides=strides,
                                 activation=None,
                                 batch_normalization=False)
            x = layers.add([x, y])
            x = layers.Activation('relu')(x)
            num_filters *= 2

    # Add classifier on top.
    x = layers.AveragePooling2D(pool_size=8)(x)
    x = layers.Flatten()(x)

    if dropout:
        x = layers.Dropout(dropout)(x)

    if classes > 2:
        outputs = layers.Dense(classes, activation="softmax", kernel_initializer='he_normal')(x)
    else:
        outputs = layers.Dense(1, activation="sigmoid")(x)

    # Instantiate model.
    model = Model(inputs=inputs, outputs=outputs, name=f"resnet_v1_depth_{depth}")
    return model

```

resnet_tl.py Code:

This script implements a transfer learning pipeline that adapts an STMicroelectronics ResNet model for MNIST digit recognition. It begins by preprocessing the standard 28x28 grayscale MNIST images into the 32x32 RGB format required by the architecture. The execution flow attempts to download pretrained weights from the ST Model Zoo (originally trained on CIFAR-10) to give the model a functional head start. If successful, it "freezes" the initial layers to preserve learned features while fine-tuning the later layers; otherwise, it defaults to training from scratch. The process concludes by compiling the model with the Adam optimizer and saving the final trained weights for deployment.

Code:

```
import os
import tensorflow as tf
from tensorflow.keras.utils import get_file, to_categorical
from tensorflow.keras.datasets import mnist
import numpy as np

# Import the model architecture
from resnet import ResNet

# -----
# Configuration and Paths
# -----
CURRENT_DIR = os.path.dirname(os.path.abspath(__file__))
# DATASET_DIR is kept for reference
DATASET_DIR = os.path.join(CURRENT_DIR, "dataset")
MODEL_FILE = os.path.join(CURRENT_DIR, "model/resnet_mnist.h5")

# Create dataset directory if it doesn't exist
if not os.path.exists(DATASET_DIR):
    os.makedirs(DATASET_DIR)

# Model Parameters
NUM_CLASSES = 10
DATA_SHAPE = (32, 32, 3) # ST Model expects 32x32 RGB
BATCH_SIZE = 32
EPOCHS = 5

# -----
# Data Preparation Functions
# -----
def prepare_tensor(images, out_shape):
    """
    Resizes images to 32x32, converts to RGB, and normalizes pixel values.
    """
    # Add channel dimension (28, 28) -> (28, 28, 1)
    images = tf.expand_dims(images, axis=-1)

    # Resize to (32, 32)
    images = tf.image.resize(images, out_shape[:2])

    # Convert Grayscale to RGB (1 channel -> 3 channels)
```

```

images = tf.repeat(images, 3, axis=-1)

# Normalize to [0.0, 1.0]
images = images / 255.0

return images

# -----
# Main Execution Flow
# -----
if __name__ == "__main__":
    print("[INFO] Loading and preparing MNIST dataset...")

    # Load Data using default cache
    (train_images, train_labels), (test_images, test_labels) = mnist.load_data()

    # Preprocess Data
    train_images = prepare_tensor(train_images, DATA_SHAPE)
    test_images = prepare_tensor(test_images, DATA_SHAPE)

    # One-hot encoding for labels
    train_labels = to_categorical(train_labels, NUM_CLASSES)
    test_labels = to_categorical(test_labels, NUM_CLASSES)

    # -----
    # Model Setup
    # -----
    print("[INFO] Initializing ResNet model...")
    # Using parameters: depth=8, dropout=0.15
    model = ResNet(input_shape=DATA_SHAPE, classes=NUM_CLASSES, depth=8, dropout=0.15)

    # -----
    # Transfer Learning Logic (New URL)
    # -----
    # Updated URL provided by you (converted to raw/main for downloading)
    st_weights_url = "https://github.com/STMicroelectronics/stm32ai-
modelzoo/raw/main/image_classification/resnetv1/ST_pretrainedmodel_public_dataset/cifar10/resnet_v1_8_32_tfs/r
esnet_v1_8_32_tfs.h5"

    print("[INFO] Attempting to download/load pretrained weights...")

    try:
        # Try to download weights
        # We save it as 'resnet_v1_8_32_new.h5' to avoid conflict with old cached files
        pretrained_model_path = get_file(fname="resnet_v1_8_32_new.h5", origin=st_weights_url,
cache_subdir="models")

        # Load weights
        model.load_weights(pretrained_model_path, by_name=True, skip_mismatch=True)
        print("[INFO] Pretrained weights loaded successfully.")

        # Freeze initial layers for Transfer Learning
        num_layers_to_train = len(model.layers) // 3
        for layer in model.layers[:num_layers_to_train]:
            layer.trainable = False

    except Exception as e:
        print(f"[WARNING] Could not download or load pretrained weights.")

```

```

print(f"[INFO] Detailed error: {e}")
print(f"[INFO] Switching to Standard Training (Training from Scratch).")

for layer in model.layers:
    layer.trainable = True

# Compile Model
model.compile(loss="categorical_crossentropy", optimizer="adam", metrics=["accuracy"])
model.summary()

# -----
# Training
# -----
print(f"[INFO] Starting training...")
model.fit(
    train_images,
    train_labels,
    epochs=EPOCHS,
    batch_size=BATCH_SIZE,
    validation_data=(test_images, test_labels)
)

# Save Model
model.save(MODEL_FILE)
print(f"[INFO] Model saved to: {MODEL_FILE}")

```

convert_resnet.py Code:

This script provides a deployment utility that converts a trained Keras model into a format suitable for embedded systems like STM32 or Mbed. It loads the H5 model file, utilizes the TFLite Converter to transform the high-level neural network into a compact flatbuffer format, and then parses that binary data into a C++ header file. By representing the model as a constant unsigned char array with 8-byte alignment, the code allows the machine learning model to be compiled directly into the microcontroller's flash memory, enabling on-device inference without the need for a file system.

Code:

```

import os
import tensorflow as tf

# -----
# Configuration and Paths
# -----
CURRENT_DIR = os.path.dirname(os.path.abspath(__file__))
MODEL_PATH = os.path.join(CURRENT_DIR, "model/resnet_mnist.h5")
OUTPUT_CC_PATH = os.path.join(CURRENT_DIR, "../hdr_cnn.cpp")

# -----
# C Array Generation Function
# -----
def convert_to_c_array(tflite_blob, filename):
    """
    Writes the TFLite binary blob to a C++ source file as a byte array.
    """

```

```

hex_array = []
for i, val in enumerate(tflite_blob):
    hex_array.append(f"0x{val:02x}")

c_code = (
    '#include "hdr_cnn.h"\n\n'
    f'// Model Size: {len(tflite_blob)} bytes\n'
    f'const unsigned int converted_model_tflite_len = {len(tflite_blob)};\n'
    f'const unsigned char converted_model_tflite[] __attribute__((aligned(8))) = {{"'
)

# Write in chunks of 12 bytes per line
for i in range(0, len(hex_array), 12):
    line = ", ".join(hex_array[i:i+12])
    c_code += " " + line + ",\n"

c_code += "};\n"

with open(filename, 'w') as f:
    f.write(c_code)
print(f"[SUCCESS] C file generated: {filename}")

# -----
# Main Execution
# -----
if __name__ == "__main__":
    if not os.path.exists(MODEL_PATH):
        print(f"[ERROR] Model file not found at {MODEL_PATH}. Please run resnet_tfl.py first.")
        exit(1)

    print("[INFO] Loading Keras model...")
    model = tf.keras.models.load_model(MODEL_PATH)

    # Initialize TFLite Converter
    converter = tf.lite.TFLiteConverter.from_keras_model(model)

    print("[INFO] Converting to TFLite with quantization...")
    tflite_model = converter.convert()

    # Convert to C Array for Mbed
    print("[INFO] Generating C array...")
    convert_to_c_array(tflite_model, OUTPUT_CC_PATH)

```

main.cpp Code:

This C++ code serves as the firmware entry point for running real-time inference on an Mbed-enabled microcontroller. It initializes the ResNet model using the converted C array and enters a continuous loop that waits for raw image data over a serial (UART) connection. Once exactly 3072 bytes (a 32x32 RGB image) are received, the firmware normalizes the pixel values to a [0, 1] float range and executes the neural network. Finally, it identifies the digit with the highest probability and transmits a single byte back to the host computer representing the predicted class.

Code:

```

#include "mbed.h"
#include "lib_model.h"
#include "hdr_cnn.h"

// -----
// CONFIGURATION (Matches resnet_tfl.py DATA_SHAPE)
// -----

const int IMG_WIDTH = 32;
const int IMG_HEIGHT = 32;
const int IMG_CHANNELS = 3; // RGB
const int IMG_SIZE = IMG_WIDTH * IMG_HEIGHT * IMG_CHANNELS; // 3072 bytes

// Buffers
uint8_t rx_buffer[IMG_SIZE];
float output_data[10];

// Serial Object
static BufferedSerial pc(USBTX, USBRX);

int main()
{
    // Baud rate match
    pc.set_baud(115200);

    // 1. Initialize Model
    // We pass nullptr because memory is managed inside lib_model.cpp
    int init_status = InitModel(converted_model_tflite, nullptr, 0);

    if (init_status != MODEL_OK) {
        // Error Indicator: Fast Blink
        DigitalOut led(LED1);
        while(1) { led = !led; thread_sleep_for(100); }
    }

    // 2. Main Loop
    while (true)
    {
        // A. Read raw image data (Blocking)
        ssize_t total_bytes_read = 0;
        while (total_bytes_read < IMG_SIZE) {
            if (pc.readable()) {
                total_bytes_read += pc.read(rx_buffer + total_bytes_read, IMG_SIZE - total_bytes_read);
            }
        }

        // B. Preprocess (Uint8 -> Float)
        // Normalize [0-255] to [0.0-1.0] matching resnet_tfl.py logic
        static float input_float_buffer[IMG_SIZE];
        for (int i=0; i<IMG_SIZE; i++) {
            input_float_buffer[i] = (float)rx_buffer[i] / 255.0f;
        }

        // C. Run Inference
        RunInference(input_float_buffer, output_data);

        // D. Find Max Probability
        float max_score = -1.0f;
        int8_t predicted_class = -1;
    }
}

```



```

        for (int i = 0; i < 10; i++) {
            if (output_data[i] > max_score) {
                max_score = output_data[i];
                predicted_class = (int8_t) i;
            }
        }

        // E. Send Result (1 Byte ONLY)
        pc.write(&predicted_class, 1);
    }
}

```

EfficientNet:

efficientnet.py Code:

This code implements a highly optimized EfficientNet architecture tailored for resource-constrained environments like STM32 microcontrollers. It utilizes MBConv (Mobile Inverted Bottleneck) blocks, which combine depthwise separable convolutions with Squeeze-and-Excitation (SE) modules to focus on important features while keeping the parameter count low. The script includes custom scaling functions (`round_filters`, `round_repeats`) to adjust the network's width and depth according to a specific coefficient, ensuring the model remains computationally efficient. Key architectural details include a "Stem" for initial feature extraction, multiple scaled stages with varying kernel sizes and expansion factors, and a classification "Head" that produces the final prediction.

Code:

```

import tensorflow as tf
import keras
from keras import layers, ops
from typing import List, Tuple
import math
from keras.models import Model

# Default EfficientNet repeats
repeats = [1, 2, 2, 3, 3, 4, 1]

def round_expansion(expansion_factor: int, repeats: List[int]) -> List[int]:
    exp_ratio = []
    flag = 1
    for r in repeats:
        if (r != 0) and flag:
            exp_ratio.append(1)
            flag = 0
        else:
            exp_ratio.append(expansion_factor)
    return exp_ratio

def round_filters(filters: int, width_coefficient: float, depth_divisor: int = 8, min_filters: int = None) -> int:
    if not width_coefficient:
        return filters
    filters *= width_coefficient
    min_filters = min_filters or depth_divisor
    new_filters = max(int(filters + depth_divisor / 2) // depth_divisor * depth_divisor, min_filters)
    if new_filters < 0.9 * filters:
        new_filters += depth_divisor

```

```

return int(new_filters)

def round_repeats(repeats: List[int]) -> List[int]:
    num_repeat = sum(repeats)
    num_repeat_scaled = int(math.ceil(num_repeat))
    repeats_scaled = []
    for r in repeats[::-1]:
        rs = max(1, round((r / num_repeat * num_repeat_scaled)))
        repeats_scaled.append(rs)
        num_repeat -= r
        num_repeat_scaled -= rs
    repeats_scaled = repeats_scaled[::-1]
    return repeats_scaled

def mb_conv_block(inputs, in_channels, out_channels, num_repeat, stride, expansion_factor, se_ratio, k,
drop_rate, prev_block_num, activation):
    x = inputs
    input_filters = in_channels

    for i in range(num_repeat):
        input_tensor = x
        current_stride = stride if i == 0 else 1
        expanded_filters = input_filters * expansion_factor

        # Expansion Phase
        if expansion_factor != 1:
            x = layers.Conv2D(filters=expanded_filters, kernel_size=(1, 1), strides=1, padding='same',
use_bias=False)(x)
            x = layers.BatchNormalization()(x)
            x = layers.Activation(activation)(x)

        # Depthwise Phase
        x = layers.DepthwiseConv2D(kernel_size=(k, k), strides=current_stride, padding='same',
use_bias=False)(x)
        x = layers.BatchNormalization()(x)
        x = layers.Activation(activation)(x)

        # Squeeze and Excitation (SE)
        squeezed_filters = max(1, int(input_filters * se_ratio))
        se = layers.GlobalAveragePooling2D()(x)
        se = layers.Reshape((1, 1, expanded_filters))(se)
        se = layers.Conv2D(filters=squeezed_filters, kernel_size=(1, 1), padding='same',
activation=activation)(se)
        se = layers.Conv2D(filters=expanded_filters, kernel_size=(1, 1), padding='same')(se)

        # FIXED: Use ops.clip instead of manual Minimum/Maximum to handle symbolic shapes
        se = ops.clip(se, -4.0, 4.0)
        se = layers.Activation('sigmoid')(se)
        x = layers.Multiply()([x, se])

        # Output Projection
        x = layers.Conv2D(filters=out_channels, kernel_size=(1, 1), strides=1, padding='same',
use_bias=False)(x)
        x = layers.BatchNormalization()(x)

        # Skip Connection
        if current_stride == 1 and input_filters == out_channels:
            if drop_rate > 0:
                x = layers.Dropout(drop_rate)(x)
            x = layers.Add()([x, input_tensor])

        input_filters = out_channels
    return x

def EfficientNet(input_shape=(32, 32, 3), classes=10, dropout_rate=0.2, se_ratio=0.25, drop_connect_rate=0.2):
    img_input = layers.Input(shape=input_shape)
    coeff = 0.45

```

```

# Stem
x = layers.Conv2D(filters=round_filters(32, coeff), kernel_size=(3, 3), strides=(2, 2), padding='same',
use_bias=False)(img_input)
x = layers.BatchNormalization()(x)
x = layers.Activation("relu6")(x)

# Scaled Blocks
r_scaled = round_repeats(repeats)
e_ratio = round_expansion(3, r_scaled)

# Blocks 1-7
x = mb_conv_block(x, round_filters(32, coeff), round_filters(16, coeff), r_scaled[0], 1, e_ratio[0],
se_ratio, 3, drop_connect_rate, 0, "relu6")
x = mb_conv_block(x, round_filters(16, coeff), round_filters(24, coeff), r_scaled[1], 2, e_ratio[1],
se_ratio, 3, drop_connect_rate, sum(r_scaled[:1]), "relu6")
x = mb_conv_block(x, round_filters(24, coeff), round_filters(40, coeff), r_scaled[2], 2, e_ratio[2],
se_ratio, 5, drop_connect_rate, sum(r_scaled[:2]), "relu6")
x = mb_conv_block(x, round_filters(40, coeff), round_filters(80, coeff), r_scaled[3], 2, e_ratio[3],
se_ratio, 3, drop_connect_rate, sum(r_scaled[:3]), "relu6")
x = mb_conv_block(x, round_filters(80, coeff), round_filters(112, coeff), r_scaled[4], 1, e_ratio[4],
se_ratio, 5, drop_connect_rate, sum(r_scaled[:4]), "relu6")
x = mb_conv_block(x, round_filters(112, coeff), round_filters(192, coeff), r_scaled[5], 2, e_ratio[5],
se_ratio, 5, drop_connect_rate, sum(r_scaled[:5]), "relu6")
x = mb_conv_block(x, round_filters(192, coeff), round_filters(320, coeff), r_scaled[6], 1, e_ratio[6],
se_ratio, 3, drop_connect_rate, sum(r_scaled[:6]), "relu6")

# Head
x = layers.Conv2D(filters=round_filters(1280, coeff), kernel_size=(1, 1), padding='same', use_bias=False)(x)
x = layers.BatchNormalization()(x)
x = layers.Activation("relu6")(x)

x = layers.GlobalAveragePooling2D()(x)
if dropout_rate > 0:
    x = layers.Dropout(dropout_rate)(x)
x = layers.Dense(classes, activation='softmax')(x)

return Model(img_input, x, name="EfficientNet-STM32")

```

efficientnet_t1.py Code:

This script implements a transfer learning pipeline that trains an EfficientNet model on the MNIST dataset, specifically optimized for STMicronics hardware. It begins by reshaping and normalizing grayscale MNIST digits into 32x32 RGB tensors to match the architecture's expected input. The script attempts to pull pretrained weights from the ST Model Zoo—originally optimized for the Flowers dataset—and applies a freezing strategy where the first 70% of the network's layers are made non-trainable to preserve low-level feature extraction capabilities. Finally, the model is fine-tuned using the Adam optimizer and saved for future conversion to an embedded format.

Code:

```

import os
import tensorflow as tf
import numpy as np
from tensorflow.keras.utils import get_file, to_categorical
from tensorflow.keras.datasets import mnist

# Import architecture from efficientnet.py
from efficientnet import EfficientNet

# Configuration

```

```

CURRENT_DIR = os.path.dirname(os.path.abspath(__file__))
MODEL_DIR = os.path.join(CURRENT_DIR, "model")
MODEL_FILE = os.path.join(MODEL_DIR, "efficientnet_mnist.h5")

if not os.path.exists(MODEL_DIR):
    os.makedirs(MODEL_DIR)

# Hyperparameters
NUM_CLASSES = 10
DATA_SHAPE = (32, 32, 3)
BATCH_SIZE = 64
EPOCHS = 10

def prepare_data():
    print("[INFO] Loading and preprocessing MNIST...")
    (x_train, y_train), (x_test, y_test) = mnist.load_data()

    def preprocess(images):
        # Resize and convert to RGB
        images = tf.expand_dims(images, axis=-1)
        images = tf.image.resize(images, [32, 32])
        images = tf.image.grayscale_to_rgb(images)
        images = images / 255.0
        return images.numpy()

    return preprocess(x_train), to_categorical(y_train, 10), preprocess(x_test), to_categorical(y_test, 10)

if __name__ == "__main__":
    x_train, y_train, x_test, y_test = prepare_data()

    print("[INFO] Initializing EfficientNet...")
    # The build error happened here; the fix is now inside efficientnet.py
    model = EfficientNet(input_shape=DATA_SHAPE, classes=NUM_CLASSES)

    # STM32 AI Model Zoo Pretrained Weights
    st_url = "https://github.com/STMicroelectronics/stm32ai-modelzoo/raw/main/image_classification/efficientnet/ST_pretrainedmodel_public_dataset/flowers/st_efficientnet_lc_v1_128_tfs/st_efficientnet_lc_v1_128_tfs.h5"

    print("[INFO] Downloading STM pretrained weights...")
    try:
        weights_path = get_file("st_effnet_pretrained.h5", st_url, cache_subdir="models")
        model.load_weights(weights_path, by_name=True, skip_mismatch=True)
        print("[INFO] Successfully loaded pretrained weights.")

        # Freeze initial 70% of layers
        freeze_until = int(len(model.layers) * 0.7)
        for layer in model.layers[:freeze_until]:
            layer.trainable = False
    except Exception as e:
        print(f"[WARNING] Weight transfer failed: {e}. Training from scratch.")

    model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])

    print("[INFO] Starting training...")
    model.fit(x_train, y_train, batch_size=BATCH_SIZE, epochs=EPOCHS, validation_data=(x_test, y_test))

    model.save(MODEL_FILE)
    print(f"[SUCCESS] Model saved to {MODEL_FILE}")

```

convert_efficientnet.py Code:

This script provides a deployment-ready conversion utility that transforms the trained EfficientNet model into a Full Integer Quantized (INT8) format for maximum efficiency on

microcontrollers. Unlike standard conversion, it uses a representative dataset from MNIST to calibrate the model's weights and activations, shrinking the model size significantly and enabling the use of hardware-accelerated integer instructions. The process concludes by generating a C++ header and source file via tflite2cc, allowing the quantized model to be embedded directly into the firmware of an edge device.

Code:

```
import os
import tensorflow as tf
import numpy as np
from tensorflow import keras
from tflite2cc import convert_tflite2cc

# Paths
CURRENT_DIR = os.path.dirname(os.path.abspath(__file__))
MODEL_PATH = os.path.join(CURRENT_DIR, "model/efficientnet_mnist.h5")
OUTPUT_CC_PATH = os.path.join(CURRENT_DIR, "../hdr_cnn")

# Load MNIST for calibration data (100 images are sufficient)
(_, _), (x_test, _) = tf.keras.datasets.mnist.load_data()

def representative_dataset():
    for i in range(100):
        # Must match training preprocessing exactly
        img = x_test[i]
        img = np.expand_dims(img, axis=-1)
        img = tf.image.resize(img, [32, 32])
        img = tf.image.grayscale_to_rgb(img)
        img = img / 255.0
        yield [np.expand_dims(img, axis=0).astype(np.float32)]

if __name__ == "__main__":
    print("[INFO] Loading Keras model...")
    model = tf.keras.models.load_model(MODEL_PATH)

    converter = tf.lite.TFLiteConverter.from_keras_model(model)

    # Enable Full Integer Quantization
    converter.optimizations = [tf.lite.Optimize.DEFAULT]
    converter.representative_dataset = representative_dataset
    converter.target_spec.supported_ops = [tf.lite.OpsSet.TFLITE_BUILTINS_INT8]

    # Force Int8 Input and Output
    converter.inference_input_type = tf.int8
    converter.inference_output_type = tf.int8

    print("[INFO] Converting to TFLite (Int8)...")
    tflite_model = converter.convert()

    # Generate C++ files
    convert_tflite2cc(tflite_model, OUTPUT_CC_PATH)
    print(f"[SUCCESS] Int8 Model Size: {len(tflite_model)/1024:.2f} KB")
```

main.cpp Code:

This C++ firmware executes quantized INT8 inference on an Mbed-enabled microcontroller, specifically optimized for the EfficientNet model. Unlike previous versions that used floating-point math, this code maps incoming pixel data from an unsigned range [0, 255] directly to a signed 8-bit integer range [-128, 127], significantly reducing the computational load and memory footprint. The main loop blocks until a full 32x32 RGB image

is received via serial, runs the inference using the RunInferenceInt8 engine, and identifies the predicted digit by finding the index with the highest integer score. This approach maximizes the hardware's performance by utilizing integer-only arithmetic, which is ideal for low-power edge AI applications.

Code:

```
#include "mbed.h"
#include "lib_model.h"
#include "hdr_cnn.h"

const int IMG_SIZE = 32 * 32 * 3;
static uint8_t rx_buffer[IMG_SIZE];
static int8_t input_int8[IMG_SIZE];
static int8_t output_int8[10];

static BufferedSerial pc(USBTX, USBRX);
DigitalOut led(LED1);

int main() {
    pc.set_baud(115200);
    thread_sleep_for(2000);

    if (InitModel(converted_model_tflite, nullptr, 0) != MODEL_OK) {
        while(1) { led = !led; thread_sleep_for(100); }
    }

    while (true) {
        ssize_t read = 0;
        while (read < IMG_SIZE) {
            if (pc.readable()) read += pc.read(rx_buffer + read, IMG_SIZE - read);
        }

        // Preprocess: Map [0, 255] to [-128, 127]
        for(int i=0; i<IMG_SIZE; i++) {
            input_int8[i] = (int8_t)((int)rx_buffer[i] - 128);
        }

        if (RunInferenceInt8(input_int8, output_int8) == MODEL_OK) {
            int8_t max_idx = 0;
            int8_t max_val = -128;
            for(int8_t i=0; i<10; i++) {
                if(output_int8[i] > max_val) { max_val = output_int8[i]; max_idx = i; }
            }
            pc.write(&max_idx, 1);
            led = !led; } } }
```

MobileNet:

mobilenet.py Code:

This code defines a MobileNetV1 architecture, a specialized neural network designed for mobile and embedded vision applications. By setting the alpha parameter (width multiplier) to 0.25, the model significantly reduces the number of filters in each layer, creating an ultra-lightweight version of the network that sacrifices a small amount of accuracy for a massive gain in speed and memory efficiency. The architecture relies on depthwise separable convolutions to decompose standard convolutions into a spatial filtering step and a feature combination step. The script removes the original ImageNet top and replaces it with a custom classification head consisting of Global Average Pooling, a dropout layer for regularization, and a Softmax output for multi-class prediction.Code:

```

import tensorflow as tf
from tensorflow.keras.applications import MobileNet
from tensorflow.keras import layers, models

def BuildMobileNetV1(input_shape, num_classes, alpha=0.25, dropout=0.2):
    """
    Builds a MobileNetV1 architecture.
    """
    # Base Model: MobileNet V1
    # We use standard Keras implementation to ensure layer naming matches the ST model
    base_model = MobileNet(
        input_shape=input_shape,
        alpha=alpha,
        depth_multiplier=1,
        dropout=dropout,
        include_top=False,
        weights=None
    )

    # Classification Head
    x = base_model.output
    x = layers.GlobalAveragePooling2D()(x)
    if dropout > 0:
        x = layers.Dropout(dropout)(x)
    outputs = layers.Dense(num_classes, activation='softmax')(x)

    # Final Model
    model = models.Model(inputs=base_model.input, outputs=outputs, name=f"mobilenet_v1_{alpha}")
    return model

```

mobilenet_tl.py Code:

This script executes a robust weight transplant and fine-tuning process to adapt a high-resolution MobileNetV1 from the ST Model Zoo for small-scale MNIST digit recognition. It includes a custom Keras patch (FixedDepthwiseConv2D) to ensure compatibility between different library versions by stripping out deprecated arguments. The pipeline downloads a pretrained model (designed for 96x96 images), builds a new target model optimized for 32x32 inputs, and programmatically "transplants" weights between layers with matching names and shapes. By freezing these inherited layers and training only the new classification head on MNIST data, the script leverages powerful pretrained features while drastically reducing training time and computational cost.

Code:

```

import os
import tensorflow as tf
from tensorflow.keras.utils import get_file, to_categorical
from tensorflow.keras.datasets import mnist
from tensorflow.keras.models import load_model
from tensorflow.keras.layers import DepthwiseConv2D
from mobilenet import BuildMobileNetV1

# -----
# KERAS VERSION PATCH

```

```

# -----
class FixedDepthwiseConv2D (DepthwiseConv2D):
    """
    Patch to ignore 'groups=1' argument from old Keras models.
    """
    def __init__(self, **kwargs):
        if 'groups' in kwargs:
            kwargs.pop('groups')
        super().__init__(**kwargs)

# -----
# CONFIGURATION
# -----
CURRENT_DIR = os.path.dirname(os.path.abspath(__file__))
MODEL_SAVE_PATH = os.path.join(CURRENT_DIR, "model/hdr_cnn.h5")

NUM_CLASSES = 10
DATA_SHAPE = (32, 32, 3)
ALPHA = 0.1

ST_MODEL_URL = "https://github.com/STMicroelectronics/stm32ai-
modelzoo/raw/main/image_classification/mobilenetv1/ST_pretrainedmodel_public_dataset/flowers/mobilenet_v1_0.25
_96_tfs/mobilenet_v1_0.25_96_tfs.h5"

# -----
# 1. PREPARE DATA
# -----
print("[INFO] Loading MNIST...")
(x_train, y_train), (x_test, y_test) = mnist.load_data()

def preprocess(images):
    images = tf.expand_dims(images, axis=-1)
    images = tf.repeat(images, 3, axis=-1)
    images = tf.image.resize(images, DATA_SHAPE[:2])
    images = images / 255.0
    return images

# Using subset for speed
x_train = preprocess(x_train[:10000])
y_train = to_categorical(y_train[:10000], NUM_CLASSES)
x_test = preprocess(x_test[:1000])
y_test = to_categorical(y_test[:1000], NUM_CLASSES)

# -----
# 2. DOWNLOAD & LOAD ST MODEL (WITH COMPILE=FALSE)
# -----
print("[INFO] Downloading ST Model Zoo weights...")
st_model_path = get_file("mobilenet_v1_0.25_96.h5", ST_MODEL_URL, cache_subdir="models")

print("[INFO] Loading Source Model (96x96)...")

# --- FIX IS HERE: compile=False ---
# We disable compilation to avoid 'reduction=auto' error.
# We only need the weights, not the optimizer/loss state.
source_model = load_model(
    st_model_path,
    custom_objects={'DepthwiseConv2D': FixedDepthwiseConv2D},
    compile=False

```



```

)
print("[INFO] Source model loaded successfully!")

# -----
# 3. TRANSFER WEIGHTS
# -----
print(f"[INFO] Building Target Model (32x32)...")
target_model = BuildMobileNetV1(DATA_SHAPE, NUM_CLASSES, alpha=ALPHA)

print("[INFO] Transplanting weights...")
transferred_count = 0
for target_layer in target_model.layers:
    try:
        # Find layer by name
        source_layer = source_model.get_layer(target_layer.name)

        # Check weights compatibility
        if source_layer.weights and target_layer.weights:
            # Check shape of the main kernel (index 0)
            if source_layer.get_weights()[0].shape == target_layer.get_weights()[0].shape:
                target_layer.set_weights(source_layer.get_weights())
                target_layer.trainable = False
                transferred_count += 1
    except:
        continue

print(f"[INFO] Transferred weights for {transferred_count} layers.")

# -----
# 4. FINE TUNE & SAVE
# -----
target_model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])

print("[INFO] Training..")
target_model.fit(x_train, y_train, epochs=5, batch_size=32, validation_data=(x_test, y_test))

target_model.save(MODEL_SAVE_PATH)
print(f"[SUCCESS] Model saved to: {MODEL_SAVE_PATH}")

```

convert_lite.py Code:

This script provides a streamlined deployment tool that converts a trained MobileNetV1 model into a Float32 TFLite format for embedded use. It loads the Keras H5 file, uses the TFLite Converter to generate a compact binary flatbuffer, and then utilizes the tflite2cc utility to transform that binary into a C++ source file. This allows the model to be compiled directly into a microcontroller's flash memory as a byte array, enabling high-performance inference on edge devices that lack a file system but possess sufficient floating-point processing power.

Code:

```

import os
import tensorflow as tf
from tensorflow import keras
from tflite2cc import convert_tflite2cc

```

```

# Paths
CURRENT_DIR = os.path.dirname(os.path.abspath(__file__))
MODEL_PATH = os.path.join(CURRENT_DIR, "model/hdr_cnn.h5")
TFLITE_MODEL_PATH = os.path.join(CURRENT_DIR, "model/hdr_cnn.tflite")
TFLITE_EXPORT_PATH = os.path.join(CURRENT_DIR, "../hdr_cnn")

# 1. Load
print(f"[INFO] Loading model from {MODEL_PATH}")
model = keras.models.load_model(MODEL_PATH)

# 2. Convert (Float32)
print("[INFO] Converting to TFLite (Float32)...")
converter = tf.lite.TFLiteConverter.from_keras_model(model)
tflite_model = converter.convert()

# 3. Save & Report
with open(TFLITE_MODEL_PATH, "wb") as f:
    f.write(tflite_model)

size_kb = len(tflite_model) / 1024
print(f"[INFO] Model Size: {size_kb:.2f} KB")

# 4. C++ Generation
convert_tflite2cc(tflite_model, TFLITE_EXPORT_PATH)
print("[SUCCESS] C++ file generated.")

```

main.cpp Code:

This C++ code implements a floating-point inference firmware for Mbed-supported microcontrollers, designed to run the MobileNetV1 model. It establishes a high-speed serial communication link at 115200 baud to receive raw 32x32 RGB image data. Upon receiving a full frame, the firmware normalizes the 8-bit pixel values into a [0.0, 1.0] floating-point range before feeding them into the inference engine. The program then identifies the digit with the highest confidence score and transmits the resulting class index as a single byte back to the host, providing a continuous loop of real-time image recognition.

Code:

```

#include "mbed.h"
#include "lib_model.h"
#include "hdr_cnn.h"

// -----
// CONFIGURATION
// -----
// Image Dimensions: 32x32 RGB => 32 * 32 * 3 = 3072 bytes
const int IMG_WIDTH = 32;
const int IMG_HEIGHT = 32;
const int IMG_CHANNELS = 3;
const int IMG_SIZE = IMG_WIDTH * IMG_HEIGHT * IMG_CHANNELS;

// Buffers
// rx_buffer: Received raw bytes (0-255) from PC via UART

```

```

uint8_t rx_buffer[IMG_SIZE];
// output_data: Probabilities for 10 digits
float output_data[10];

// Serial Object (USBTX/RX connects to ST-Link Virtual COM Port)
static BufferedSerial pc(USBTX, USBRX);

int main()
{
    // Set baud rate to match Python script (115200)
    pc.set_baud(115200);

    // 1. Initialize Model
    // We pass nullptr because memory is managed inside lib_model.cpp
    int init_status = InitModel(converted_model_tflite, nullptr, 0);

    if (init_status != MODEL_OK) {
        // Blink LED fast to indicate initialization error
        DigitalOut led(LED1);
        while(1) { led = !led; thread_sleep_for(100); }
    }

    // 2. Main Loop: Wait for Data -> Predict -> Send Result
    while (true)
    {
        // A. Read raw image data from UART (Blocking call)
        // We wait until exactly 3072 bytes are received
        ssize_t total_bytes_read = 0;
        while (total_bytes_read < IMG_SIZE) {
            if (pc.readable()) {
                total_bytes_read += pc.read(rx_buffer + total_bytes_read, IMG_SIZE - total_bytes_read);
            }
        }

        // B. Preprocess Data (Uint8 [0-255] -> Float [0.0-1.0])
        // We convert the raw bytes to normalized floats for the model
        static float input_float_buffer[IMG_SIZE];
        for(int i=0; i<IMG_SIZE; i++) {
            input_float_buffer[i] = (float)rx_buffer[i] / 255.0f;
        }

        // C. Run Inference
        RunInference(input_float_buffer, output_data);

        // D. Find Max Probability Class
        float max_score = -1.0f;
        int8_t predicted_class = -1; // int8_t to send as a single byte

        for (int i = 0; i < 10; i++) {
            if (output_data[i] > max_score) {
                max_score = output_data[i];
                predicted_class = (int8_t)i;
            }
        }

        // E. Send Result back to PC (1 Byte)
        pc.write(&predicted_class, 1);
    }
}

```

```
}
```

SqueezeNet:

squeezenet.py Code:

This code implements a highly efficient SqueezeNet architecture, specifically optimized for deployment on STM32 microcontrollers. The core of the design is the "Fire Module," which significantly reduces the parameter count by first "squeezing" the input with 1x1 convolutions and then "expanding" it with a mix of 1x1 and 3x3 filters. This version is truncated to four Fire modules to maintain a low memory footprint while still allowing for the loading of pretrained ImageNet weights. The architecture concludes with a Global Average Pooling layer and a Softmax output, making it ideal for high-speed image classification on edge devices where storage and RAM are extremely limited.

Code:

```
import tensorflow as tf
from keras.utils import get_file

WEIGHTS_PATH_NO_TOP = "https://github.com/rcmalli/keras-
squeezenet/releases/download/v1.0/squeezenet_weights_tf_dim_ordering_tf_kernels_notop.h5"

def fire(x, squeeze, expand, name):
    y = tf.keras.layers.Conv2D(filters=squeeze, kernel_size=1, activation="relu", padding="same",
name=f"{name}_s")(x)
    y1 = tf.keras.layers.Conv2D(filters=expand, kernel_size=1, activation="relu", padding="same",
name=f"{name}_e1")(y)
    y3 = tf.keras.layers.Conv2D(filters=expand, kernel_size=3, activation="relu", padding="same",
name=f"{name}_e3")(y)
    return tf.keras.layers.concatenate([y1, y3], name=f"{name}_c")

def SqueezeNet(input_shape=(32, 32, 3), weights="imagenet", classes=10, dropout=0.2):
    model_input = tf.keras.layers.Input(shape=input_shape)

    # Giriş Katmanı
    x = tf.keras.layers.Conv2D(64, (3, 3), strides=2, padding="same", activation="relu",
name="conv1")(model_input)
    x = tf.keras.layers.MaxPooling2D((3, 3), strides=2, padding="same")(x)

    # STM32 için optimize edilmiş derinlik (Fire 4'e kadar)
    x = fire(x, 16, 64, name="fire1")
    x = fire(x, 16, 64, name="fire2")
    x = tf.keras.layers.MaxPooling2D((3, 3), strides=2, padding="same")(x)
    x = fire(x, 32, 128, name="fire3")
    x = fire(x, 32, 128, name="fire4")

    if weights == "imagenet":
        weights_path = get_file("sq_weights_notop.h5", WEIGHTS_PATH_NO_TOP, cache_subdir="models")
        # Sadece mevcut katmanların ağırlıklarını yükle
        tmp_model = tf.keras.Model(inputs=model_input, outputs=x)
        tmp_model.load_weights(weights_path, by_name=True, skip_mismatch=True)

    if dropout:
        x = tf.keras.layers.Dropout(dropout)(x)

    x = tf.keras.layers.Conv2D(classes, (1, 1), padding="same", name="final_conv")(x)
    x = tf.keras.layers.GlobalAveragePooling2D(name="global_avg_pool")(x)
    model_output = tf.keras.layers.Softmax()(x)

    return tf.keras.Model(inputs=model_input, outputs=model_output)
```

squeezenetnet_tl.py Code:

This script coordinates a transfer learning workflow to train the SqueezeNet model for MNIST digit recognition. It first handles data preparation by expanding and resizing the grayscale MNIST dataset into 32x32 RGB tensors suitable for the network's input. The training logic initializes the model with pretrained weights and freezes the first third of the layers to leverage existing features while fine-tuning the remaining parameters. A ModelCheckpoint callback is integrated to monitor validation loss, ensuring that only the most accurate version of the model is saved to the local directory.

Code:

```
import os
import tensorflow as tf
from squeezenet import SqueezeNet

# Hardcoded paths replacing Models.paths
CURRENT_DIR = os.path.dirname(os.path.abspath(__file__))
KERAS_MODEL_DIR = os.path.join(CURRENT_DIR, "model")
if not os.path.exists(KERAS_MODEL_DIR): os.makedirs(KERAS_MODEL_DIR)

model_checkpoint_path = os.path.join(KERAS_MODEL_DIR, "squeezenet_tl_mnist.h5")
num_classes = 10
data_shape = (32, 32, 3)

def prepare_tensor(images, out_shape):
    images = tf.expand_dims(images, axis=-1)
    images = tf.repeat(images, 3, axis=-1)
    images = tf.image.resize(images, out_shape[:2])
    images = images / 255.0
    return images

if __name__ == "__main__":
    print("[INFO] Loading and preparing MNIST dataset...")
    (train_images, train_labels), (test_images, test_labels) = tf.keras.datasets.mnist.load_data()

    train_images = prepare_tensor(train_images, data_shape)
    test_images = prepare_tensor(test_images, data_shape)

    train_labels = tf.keras.utils.to_categorical(train_labels, num_classes)
    test_labels = tf.keras.utils.to_categorical(test_labels, num_classes)

    model = SqueezeNet(input_shape=data_shape, dropout=0.2)

    # Freeze initial layers for Transfer Learning
    num_layers_to_train = len(model.layers) // 3
    for layer in model.layers[:num_layers_to_train]:
        layer.trainable = False

    model.compile(loss="categorical_crossentropy", optimizer="adam", metrics=["accuracy"])

    callbacks = [
        tf.keras.callbacks.ModelCheckpoint(
            model_checkpoint_path,
            monitor="val_loss",
            save_best_only=True,
            mode="min",
            verbose=1,
        )
    ]
```

```

    model.fit(
        train_images,
        train_labels,
        batch_size=128,
        epochs=10,
        validation_split=0.1,
        callbacks=callbacks,
    )
    print(f"[SUCCESS] Model saved to {model_checkpoint_path}")
    while(1) { status_led = !status_led;
wait_us(1000 * 1000); } // Slow Blink Error
}

// D. PROCESS LOOP
while (true) {
    // 1. Receive Synced Frame (Waits for '$')
    Serial_ReceiveFrame((uint8_t*)input_buffer, NUM_INPUTS * sizeof(float));

    // 2. Normalize & Populate Input
    for (int i = 0; i < NUM_INPUTS; i++) {
        float normalized = (input_buffer[i] - TRAIN_MEAN) / TRAIN_STD;

        if (input->type == kTfLiteFloat32) {
            input->data.f[i] = normalized;
        }
        else if (input->type == kTfLiteInt8) {
            int32_t val = (normalized / input->params.scale) + input->params.zero_point;
            val = (val < -128) ? -128 : (val > 127 ? 127 : val);
            input->data.int8[i] = (int8_t)val;
        }
        else if (input->type == kTfLiteUInt8) {
            int32_t val = (normalized / input->params.scale) + input->params.zero_point;
            val = (val < 0) ? 0 : (val > 255 ? 255 : val);
            input->data.uint8[i] = (uint8_t)val;
        }
    }

    // 3. Inference
    if (LIB_MODEL_Run(&output) != 0 || output == nullptr) {
        float err = -999.0f;
        LIB_SERIAL_Transmit(&err, 1, TYPE_F32);
        continue;
    }

    // 4. Read Output
    float raw_output = 0.0f;

    if (output->type == kTfLiteFloat32) {
        raw_output = output->data.f[0];
    }
    else if (output->type == kTfLiteInt8) {
        raw_output = (output->data.int8[0] - output->params.zero_point) * output->params.scale;
    }
    else if (output->type == kTfLiteUInt8) {
        raw_output = (output->data.uint8[0] - output->params.zero_point) * output->params.scale;
    }

    // 5. Final Calculation
    float predicted = (raw_output * TRAIN_STD) + TRAIN_MEAN;
    LIB_SERIAL_Transmit(&predicted, 1, TYPE_F32);

    status_led = 1; wait_us(50 * 1000); status_led = 0;
}
}

```

convert_squeezenet.py Code:

This script provides a deployment utility that performs Full Integer Quantization (INT8) on the SqueezeNet model to optimize it for edge hardware. It uses a representative dataset from MNIST to calibrate the internal weights and activations, converting the model from floating-point to 8-bit integers. A unique feature of this conversion is the setting of input and output types to uint8, which simplifies data handling over UART for microcontrollers. Finally, it uses the tflite2cc tool to generate C++ source and header files, effectively turning the compressed model into a static byte array that can be compiled directly into an embedded system's memory.

Code:

```
import os
import tensorflow as tf
import numpy as np
from tflite2cc import convert_tflite2cc

# Local Path Configuration
CURRENT_DIR = os.path.dirname(os.path.abspath(__file__))
KERAS_MODEL_PATH = os.path.join(CURRENT_DIR, "model/squeezenet_t1_mnist.h5")
TFLITE_EXPORT_PATH = os.path.join(CURRENT_DIR, "../hdr_cnn") # Will generate .h and .cpp

def representative_dataset():
    (_, _), (x_test, _) = tf.keras.datasets.mnist.load_data()
    # Use 100 samples for calibration
    x_test = x_test[:100]
    x_test = tf.expand_dims(x_test, axis=-1)
    x_test = tf.repeat(x_test, 3, axis=-1)
    x_test = tf.image.resize(x_test, [32, 32])
    x_test = x_test / 255.0
    for i in range(100):
        yield [x_test[i:i+1]]

if __name__ == "__main__":
    if not os.path.exists(KERAS_MODEL_PATH):
        print(f"[ERROR] Run squeezenet_t1.py first. Missing: {KERAS_MODEL_PATH}")
        exit(1)

    hdr_cnn_model = tf.keras.models.load_model(KERAS_MODEL_PATH)
    converter = tf.lite.TFLiteConverter.from_keras_model(hdr_cnn_model)
    converter.optimizations = [tf.lite.Optimize.DEFAULT]
    converter.target_spec.supported_ops = [tf.lite.OpsSet.TFLITE_BUILTINS_INT8]
    converter.representative_dataset = representative_dataset

    # Set IO to Uint8 for easier UART handling
    converter.inference_input_type = tf.uint8
    converter.inference_output_type = tf.uint8

    print("[INFO] Converting to Full Int8 TFLite...")
    tflite_model = converter.convert()

    # Save .tflite file
    with open(KERAS_MODEL_PATH.replace(".h5", ".tflite"), "wb") as f:
        f.write(tflite_model)

    # Export to C++
    convert_tflite2cc(tflite_model, TFLITE_EXPORT_PATH)
    print(f"[SUCCESS] C++ files generated at {TFLITE_EXPORT_PATH}.cpp/h")
```

main.cpp Code:

This C++ firmware completes the deployment cycle by running the quantized SqueezeNet model on an Mbed-enabled microcontroller using highly efficient integer-only inference. Because the model was converted with uint8 inputs, the code can pass the raw serial buffer directly to the inference engine via a simple pointer cast, bypassing the need for computationally expensive normalization or floating-point conversions. The program blocks until a full image is received, executes the prediction, and transmits the resulting class index as a single byte. This implementation represents the peak of optimization for this series, offering the smallest memory footprint and the fastest execution speed for edge AI digit recognition. Code:

```
#include "mbed.h"
#include "lib_model.h"
#include "hdr_cnn.h"

// Buffers must be static to avoid stack growth
static uint8_t rx_buffer[3072];
static int8_t output_buffer[10];
static BufferedSerial pc(USBTX, USBRX);
DigitalOut led(LED1);

int main() {
    pc.set_baud(115200);

    // Initialize model without any serial logging
    if (InitModel(converted_model_tflite, nullptr, 0) != MODEL_OK) {
        // Blink fast if Init fails
        while(1) { led = !led; thread_sleep_for(100); }
    }

    while (true) {
        ssize_t total_read = 0;
        // Wait for 3072 bytes (32x32x3)
        while (total_read < 3072) {
            if (pc.readable()) {
                total_read += pc.read(rx_buffer + total_read, 3072 - total_read);
            }
        }

        // Run inference with the signed pointer cast
        if (RunInferenceInt8((int8_t*)rx_buffer, output_buffer) == MODEL_OK) {
            int8_t max_idx = 0;
            int8_t max_val = -128;

            for(int i = 0; i < 10; i++) {
                if(output_buffer[i] > max_val) {
                    max_val = output_buffer[i];
                    max_idx = (int8_t)i;
                }
            }

            // Send 1-byte result via UART
            pc.write(&max_idx, 1);
            led = !led;
        }
    }
}
```